WESTERN SYDNEY
UNIVERSITY

Introduction to Arrays

ELEC1006: ENGINEERING COMPUTING

Array

WESTERN SYDNEY UNIVERSITY

- Array: variable that can store multiple values of the same type.
- Values are stored in adjacent memory locations.
- Declared using `[]` operator:

```
int tests[5];
```

- In the definition `int tests[5];`
  - `int` is the data type of the array elements.
  - `tests` is the name of the array.
  - `5`, in `[5]`, is the size declarator. It shows the number of elements in the array.

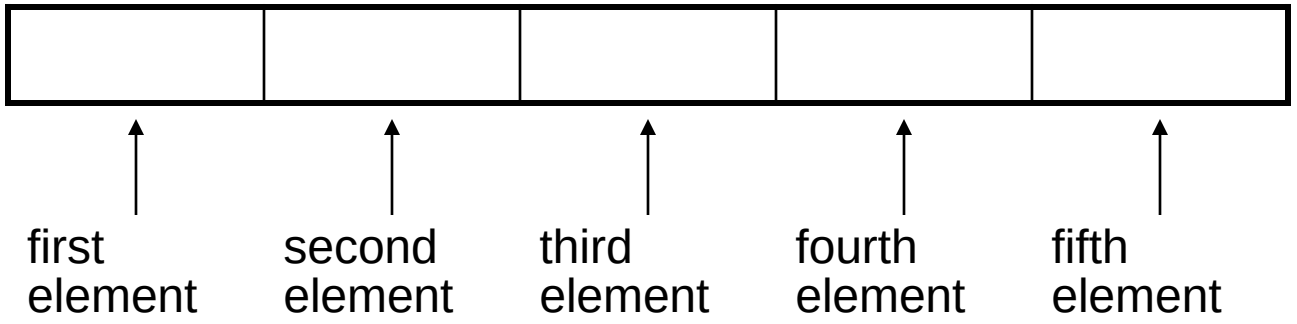
Array Memory Layout

WESTERN SYDNEY UNIVERSITY

- The definition:

```
int tests[5];
```

allocates the following memory:



Size of an Array

WESTERN SYDNEY UNIVERSITY

- The size of an array is:
  - the total number of bytes allocated for it.
  - (number of elements) * (number of bytes for each element).
- Examples:

  `int tests[5]` is an array of 20 bytes, assuming 4 bytes for an `int`.

  `long double measures[10]` is an array of 80 bytes, assuming 8 bytes for a `long double`.

Size Declarators

WESTERN SYDNEY UNIVERSITY

- Named constants are commonly used as size declarators.

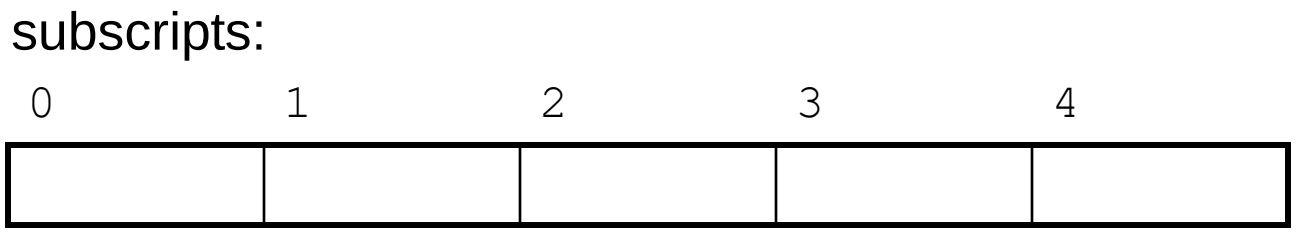
```
const int SIZE = 5;
int tests[SIZE];
```

- This eases program maintenance when the size of the array needs to be changed.

Array Subscript

WESTERN SYDNEY UNIVERSITY

- Each element in an array is assigned a unique *subscript*.
- **Subscripts start at 0.**
- **The last element's subscript is *n*-1 where *n* is the number of elements in the array.**



Accessing Array Elements

WESTERN SYDNEY UNIVERSITY

- Array elements can be used as regular variables:

```
tests[0] = 79;
cout << tests[0];
cin >> tests[1];
tests[4] = tests[0] + tests[1];
```

- Arrays must be accessed via individual elements:

```
cout << tests; // illegal
```

Example 1

WESTERN SYDNEY UNIVERSITY

```
1  // This program asks for the number of hours worked
2  // by six employees. It stores the values in an array.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8     const int NUM_EMPLOYEES = 6;
9     int hours[NUM_EMPLOYEES];
10
11    // Get the hours worked by each employee.
12    cout << "Enter the hours worked by "
13         << NUM_EMPLOYEES << " employees: ";
14    cin >> hours[0];
15    cin >> hours[1];
16    cin >> hours[2];
17    cin >> hours[3];
18    cin >> hours[4];
19    cin >> hours[5];
20
```

Example 1 (continued)

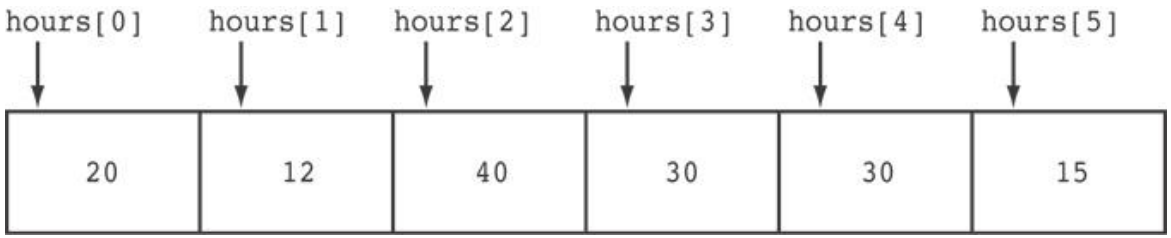
WESTERN SYDNEY UNIVERSITY

```
21    // Display the values in the array.
22    cout << "The hours you entered are:";
23    cout << " " << hours[0];
24    cout << " " << hours[1];
25    cout << " " << hours[2];
26    cout << " " << hours[3];
27    cout << " " << hours[4];
28    cout << " " << hours[5] << endl;
29    return 0;
30 }
```

Program Output with Example Input Shown in Bold

```
Enter the hours worked by 6 employees: 20 12 40 30 30 15 [Enter]
The hours you entered are: 20 12 40 30 30 15
```

Here are the contents of the `hours` array, with the values entered by the user in the example output:



More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] learncpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

WESTERN SYDNEY
UNIVERSITY

Accessing Array Contents

ELEC1006 ENGINEERING COMPUTING

Accessing Array Contents

WESTERN SYDNEY UNIVERSITY

- Can access element with a constant or literal subscript:

```
cout << tests[3] << endl;
```

- Can use integer expression as subscript:

```
int i = 5;
cout << tests[i] << endl;
```

Using a Loop to Step Through an Array

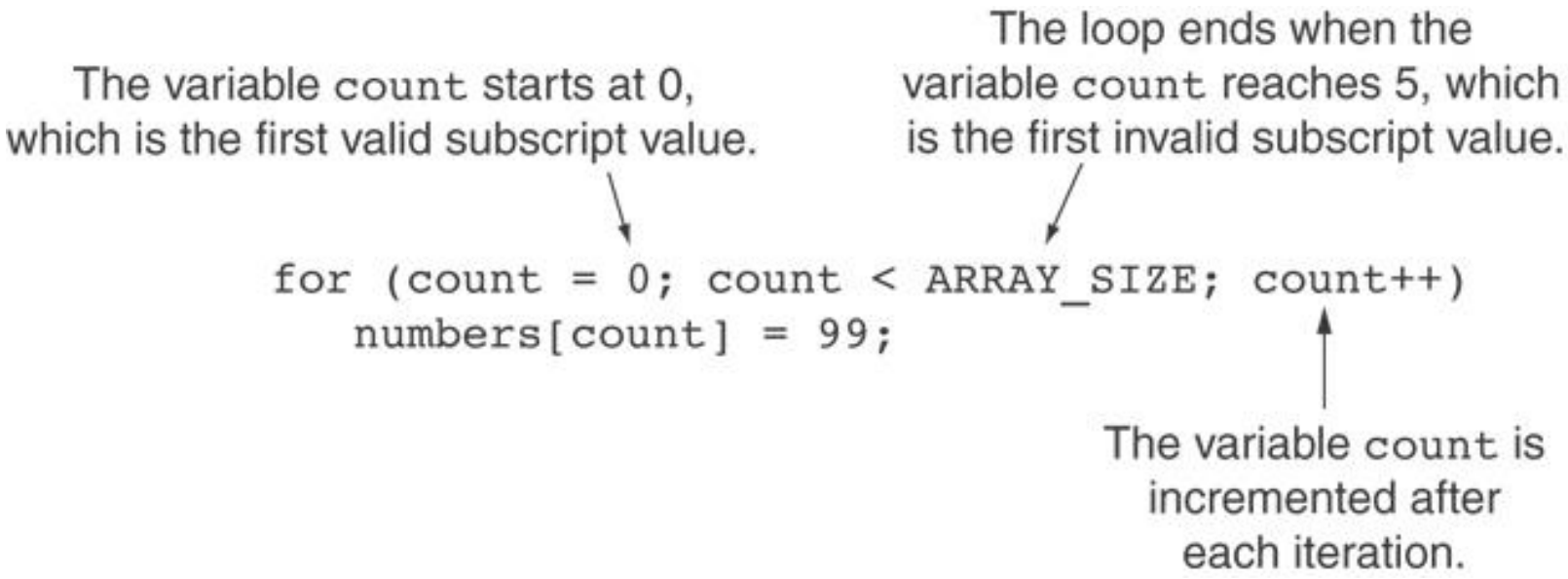
WESTERN SYDNEY UNIVERSITY

- Example – The following code defines an array, `numbers`, and assigns 99 to each element:

```
const int ARRAY_SIZE = 5;
int numbers[ARRAY_SIZE];

for (int count = 0; count < ARRAY_SIZE; count++)
     numbers[count] = 99;
```

A Closer Look at the Loop

WESTERN SYDNEY UNIVERSITY



Default Initialisation

WESTERN SYDNEY UNIVERSITY

- Global array → all elements initialized to `0` by default

- Local array → all elements *uninitialized* by default

No Bounds Checking in C++

WESTERN SYDNEY UNIVERSITY

- When you use a value as an array subscript, C++ does not check it to make sure it is a *valid* subscript.

- In other words, you can use subscripts that are beyond the bounds of the array.

Example 2

WESTERN SYDNEY UNIVERSITY

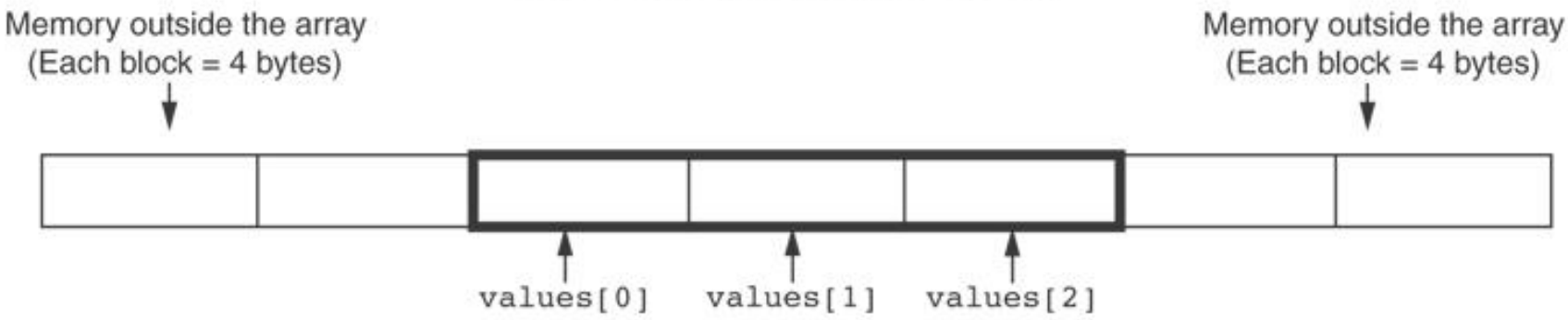
- The following code defines a three-element array, and then writes five values to it!

```
 9     const int SIZE = 3;   // Constant for the array size
10     int values[SIZE];     // An array of 3 integers
11     int count;            // Loop counter variable
12
13     // Attempt to store five numbers in the three-element array.
14     cout << "I will store 5 numbers in a 3 element array!\n";
15     for (count = 0; count < 5; count++)
16        values[count] = 100;
```

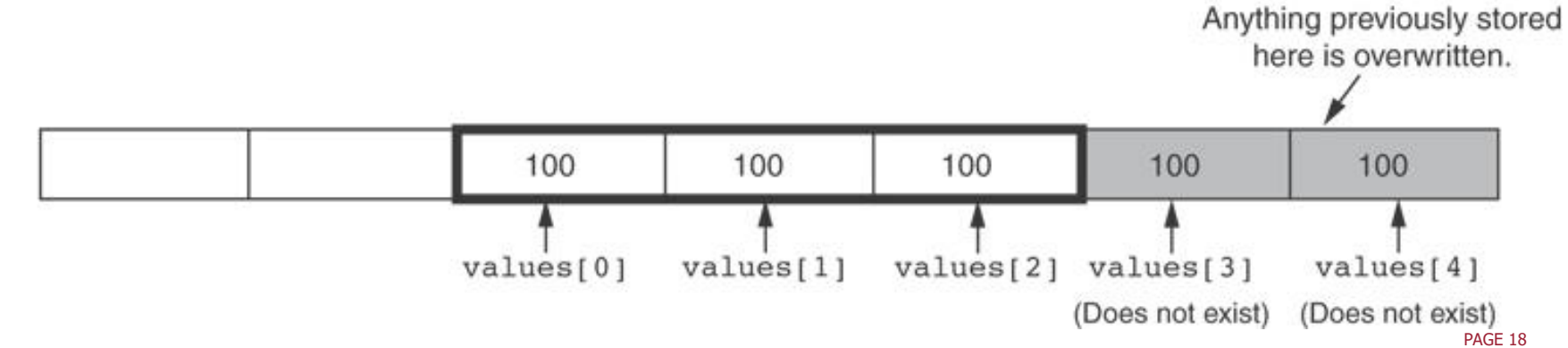
Example 2 (continued)

WESTERN SYDNEY UNIVERSITY

The way the `values` array is set up in memory.
The outlined area represents the array.



How the numbers assigned to the array overflow the array's boundaries.
The shaded area is the section of memory illegally written to.



No Bounds Checking in C++

WESTERN SYDNEY UNIVERSITY

- **Be careful not to use invalid subscripts**.
- **Doing so can corrupt other memory locations, crash program, or lock up computer, and cause elusive bugs**.

Off-by-One Errors

WESTERN SYDNEY UNIVERSITY

- **An off-by-one error happens when you use array subscripts that are off by one**.
- This can happen when you start subscripts at 1 rather than 0:

```
// This code has an off-by-one error.
const int SIZE = 100;
int numbers[SIZE];
for (int count = 1; count <= SIZE; count++)
   numbers[count] = 0;
```

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

Array Initialisation & Assignment

ELEC1006: ENGINEERING COMPUTING

Array Initialisation

WESTERN SYDNEY UNIVERSITY

- Arrays can be initialised with an initialisation list:

```
const int SIZE = 5;
int tests[SIZE] = {79,82,91,77,84};
```

- The values are stored in the array in the order in which they appear in the list.
- The initialisation list cannot exceed the array size.

Example 1

WESTERN SYDNEY UNIVERSITY

```
7     const int MONTHS = 12;
8     int days[MONTHS] = { 31, 28, 31, 30,
9                          31, 30, 31, 31,
10                         30, 31, 30, 31};
11
12    for (int count = 0; count < MONTHS; count++)
13    {
14       cout << "Month " << (count + 1) << " has ";
15       cout << days[count] << " days.\n";
16    }
```

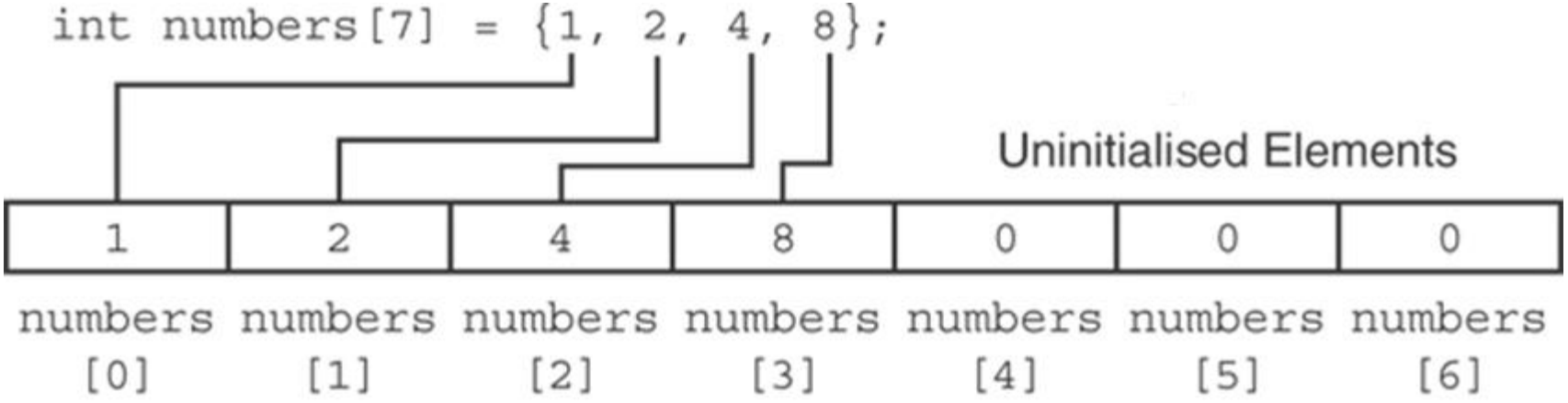
Program Output

```
Month 1 has 31 days.
Month 2 has 28 days.
Month 3 has 31 days.
Month 4 has 30 days.
Month 5 has 31 days.
Month 6 has 30 days.
Month 7 has 31 days.
Month 8 has 31 days.
Month 9 has 30 days.
Month 10 has 31 days.
Month 11 has 30 days.
Month 12 has 31 days.
```

Partial Array Initialisation

WESTERN SYDNEY UNIVERSITY

- If an array is initialised with fewer initial values than the size declarator, the remaining elements will be set to `0`:



Implicit Array Sizing

WESTERN SYDNEY UNIVERSITY

- Can determine array size by the size of the initialisation list:

```
int quizzes[]={12,17,15,11};
```

- Must use either array size declarator or initialisation list for array definition.

Initialising an Array with a String

WESTERN SYDNEY UNIVERSITY

- Character array can be initialised by enclosing string in " ":

```
const int SIZE = 6;
char fName[SIZE] = "Henry";
```

- Must leave room for \0 at end of array
- If initialising character-by-character, must add in \0 explicitly:

```
char fName[SIZE] =
{ 'H', 'e', 'n', 'r', 'y', '\0'};
```

Array Assignment

WESTERN SYDNEY UNIVERSITY

To copy one array to another,

- Don’t try to assign one array to the other:

```
newTests = tests;  // Won't work
```

- Instead, assign element-by-element:

```
for (i = 0; i < ARRAY_SIZE; i++)
   newTests[i] = tests[i];
```

Printing String Contents of an Array

WESTERN SYDNEY UNIVERSITY

- You can display the contents of a *character* array by sending its name to cout:

```
char fName[] = "Henry";
cout << fName << endl;
```

But, this ONLY works with character arrays!

Printing Numeric Contents of an Array

WESTERN SYDNEY UNIVERSITY

- For other types of arrays, you must print element-by-element:

```
for (i = 0; i < ARRAY_SIZE; i++)
   cout << tests[i] << endl;
```

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

Operations on Arrays

ELEC1006: ENGINEERING COMPUTING

Increment & Decrement Operations on an Array

WESTERN SYDNEY UNIVERSITY

- Array elements can be treated as ordinary variables of the same type as the array.

- When using ++, −− operators, don't confuse the element with the subscript:

```
tests[i]++; // add 1 to tests[i]
tests[i++]; // increment i, no
            // effect on tests
```

Summing & Averaging Array Elements

WESTERN SYDNEY UNIVERSITY

- Use a simple loop to add together array elements:

```
int tnum;
double average, sum = 0;
for(tnum = 0; tnum < SIZE; tnum++)
      sum += tests[tnum];
```

- Once summed, the average can be calculated immediately outside the loop:

```
average = sum / SIZE;
```

Finding the Highest Value in an Array

WESTERN SYDNEY UNIVERSITY

```
int count;
int highest;
highest = numbers[0];
for (count = 1; count < SIZE; count++)
{
   if (numbers[count] > highest)
      highest = numbers[count];
}
```

When this code execution is finished, the `highest` variable will contain the largest value from the `numbers` array.

Finding the Lowest Value in an Array

WESTERN SYDNEY UNIVERSITY

```
int count;
int lowest;
lowest = numbers[0];
for (count = 1; count < SIZE; count++)
{
   if (numbers[count] < lowest)
      lowest = numbers[count];
}
```

When this code is finished, the `lowest` variable will contain the smallest value in the `numbers` array.

Comparing Arrays

WESTERN SYDNEY UNIVERSITY

- To compare two arrays, you must compare element-by-element:

```
const int SIZE = 5;
int firstArray[SIZE] = { 5, 10, 15, 20, 25 };
int secondArray[SIZE] = { 5, 10, 15, 20, 25 };
bool arraysEqual = true; // Flag variable
int count = 0;           // Loop counter variable
// Compare the two arrays.
while (arraysEqual && count < SIZE)
{
   if (firstArray[count] != secondArray[count])
      arraysEqual = false;
   count++;
}
if (arraysEqual)
   cout << "The arrays are equal.\n";
else
   cout << "The arrays are not equal.\n";
```

Partially-Filled Arrays

WESTERN SYDNEY UNIVERSITY

- If it is unknown how much data an array will be holding:
  - Make the array large enough to hold the largest expected number of elements.
  - Use a counter variable to keep track of the number of items stored in the array.

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

Arrays in a Function

ELEC1006: ENGINEERING COMPUTING

Arrays as Function Arguments

WESTERN SYDNEY UNIVERSITY

- When passing an array to a function, it is common to pass array size so that a function knows how many elements to process:

```
showScores(tests, ARRAY_SIZE);
```

- Array size must also be reflected in prototype, header:

```
void showScores(int [], int);
                // function prototype
void showScores(int tests[], int size)
                // function header
```

Example 1

WESTERN SYDNEY UNIVERSITY

```
1  // This program demonstrates an array being passed to a function.
2  #include <iostream>
3  using namespace std;
4
5  void showValues(int [], int); // Function prototype
6
7  int main()
8  {
9     const int ARRAY_SIZE = 8;
10    int numbers[ARRAY_SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
11
12    showValues(numbers, ARRAY_SIZE);
13    return 0;
14 }
15
```

(Program Continues)

Example 1 (continued)

WESTERN SYDNEY UNIVERSITY

```
16  //*************************************************
17  // Definition of function showValue.              *
18  // This function accepts an array of integers and  *
19  // the array's size as its arguments. The contents *
20  // of the array are displayed.                     *
21  //*************************************************
22
23  void showValues(int nums[], int size)
24  {
25     for (int index = 0; index < size; index++)
26        cout << nums[index] << " ";
27     cout << endl;
28  }
```

Program Output

```
5 10 15 20 25 30 35 40
```

Modifying Arrays in a Function

WESTERN SYDNEY UNIVERSITY

- Array names in functions are like reference variables – changes made to array in a function are reflected in the actual array in the calling function.

- **Need to exercise caution that array is not inadvertently changed by a function.**

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

Arrays in a Function

ELEC1006 ENGINEERING COMPUTING

Arrays as Function Arguments

WESTERN SYDNEY UNIVERSITY

- When passing an array to a function, it is common to pass array size so that a function knows how many elements to process:

```
showScores(tests, ARRAY_SIZE);
```

- Array size must also be reflected in prototype, header:

```
void showScores(int [], int);
                // function prototype
void showScores(int tests[], int size)
                // function header
```

Example 1

WESTERN SYDNEY UNIVERSITY

```
1  // This program demonstrates an array being passed to a function.
2  #include <iostream>
3  using namespace std;
4
5  void showValues(int [], int); // Function prototype
6
7  int main()
8  {
9     const int ARRAY_SIZE = 8;
10    int numbers[ARRAY_SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
11
12    showValues(numbers, ARRAY_SIZE);
13    return 0;
14 }
15
```

(Program Continues)

Example 1 (continued)

WESTERN SYDNEY UNIVERSITY

```
16  //*************************************************
17  // Definition of function showValue.              *
18  // This function accepts an array of integers and  *
19  // the array's size as its arguments. The contents *
20  // of the array are displayed.                     *
21  //*************************************************
22
23  void showValues(int nums[], int size)
24  {
25     for (int index = 0; index < size; index++)
26        cout << nums[index] << " ";
27     cout << endl;
28  }
```

Program Output

```
5 10 15 20 25 30 35 40
```

Modifying Arrays in a Function

WESTERN SYDNEY UNIVERSITY

- Array names in functions are like reference variables – changes made to array in a function are reflected in the actual array in the calling function.

- **Need to exercise caution that array is not inadvertently changed by a function.**

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

2D Arrays in a Function

ELEC1006: ENGINEERING COMPUTING

2D Array Initialisation

WESTERN SYDNEY UNIVERSITY

- Two-dimensional arrays are initialised row-by-row:

```
const int ROWS = 2, COLS = 2;
int exams[ROWS][COLS] = { {84, 78},
                          {92, 97} };
```

- Can omit inner `{ }`, some initial values in a row – array elements without initial values will be set to `0` or `NULL`

2D Array as Argument & Parameter

WESTERN SYDNEY UNIVERSITY

- Use array name as argument in function call:

```
getExams(exams, 2);
```

- Use empty `[]` for row, size declarator for column in prototype, header:

```
const int COLS = 2;
// Prototype
void getExams(int [][COLS], int);

// Header
void getExams(int exams[][COLS], int rows)
```

Example 1: The `showArray` Function

WESTERN SYDNEY UNIVERSITY

```
30  //*************************************************************
31  // Function Definition for showArray                          *
32  // The first argument is a two-dimensional int array with COLS *
33  // columns. The second argument, rows, specifies the number of *
34  // rows in the array. The function displays the array's contents. *
35  //*************************************************************
36
37  void showArray(int array[][COLS], int rows)
38  {
39     for (int x = 0; x < rows; x++)
40     {
41        for (int y = 0; y < COLS; y++)
42        {
43           cout << setw(4) << array[x][y] << " ";
44        }
45        cout << endl;
46     }
47  }
```

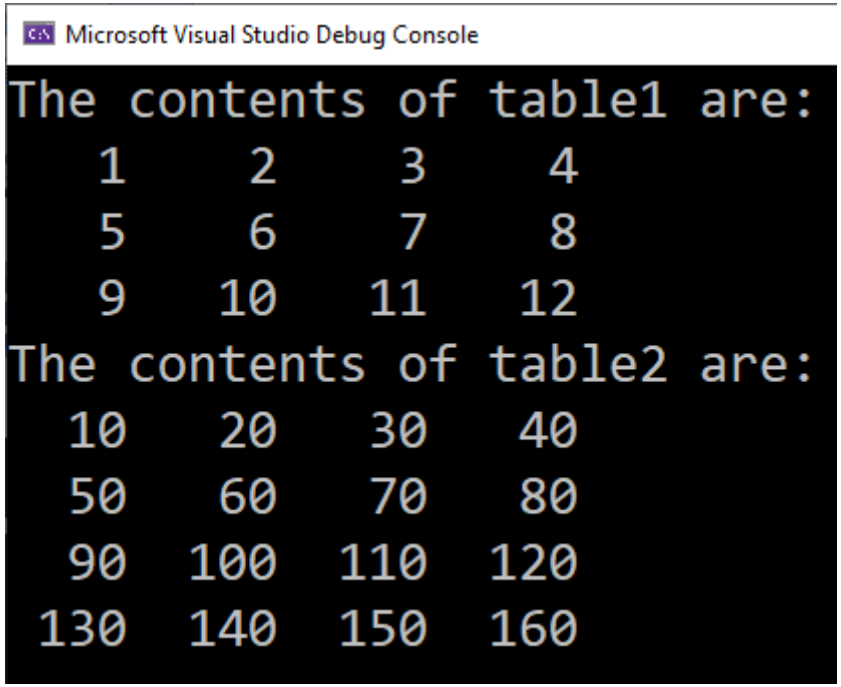
How showArray is Called

WESTERN SYDNEY UNIVERSITY

```
15      int table1[TBL1_ROWS][COLS] = {{1, 2, 3, 4},
16                                     {5, 6, 7, 8},
17                                     {9, 10, 11, 12}};
18      int table2[TBL2_ROWS][COLS] = {{10, 20, 30, 40},
19                                     {50, 60, 70, 80},
20                                     {90, 100, 110, 120},
21                                     {130, 140, 150, 160}};
22
23      cout << "The contents of table1 are:\n";
24      showArray(table1, TBL1_ROWS);
25      cout << "The contents of table2 are:\n";
26      showArray(table2, TBL2_ROWS);
```

Example 1 Output

WESTERN SYDNEY UNIVERSITY



More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

Operations on 2D Arrays

ELEC1006: ENGINEERING COMPUTING

2D Array Initialisation

WESTERN SYDNEY UNIVERSITY

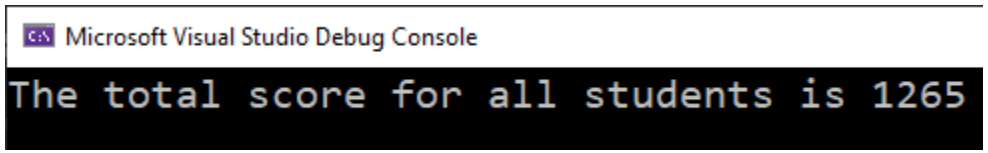
- Given the following definitions:

```
const int NUM_STUDENTS = 3;
const int NUM_SCORES = 5;
double total = 0.0;   // Accumulator
double average; // To hold average scores
double scores[NUM_STUDENTS][NUM_SCORES] =
     {{88, 97, 79, 86, 94},
      {86, 91, 78, 79, 84},
      {82, 73, 77, 82, 89}};
```

Example 1: Summing all Elements in a 2D Array

WESTERN SYDNEY UNIVERSITY

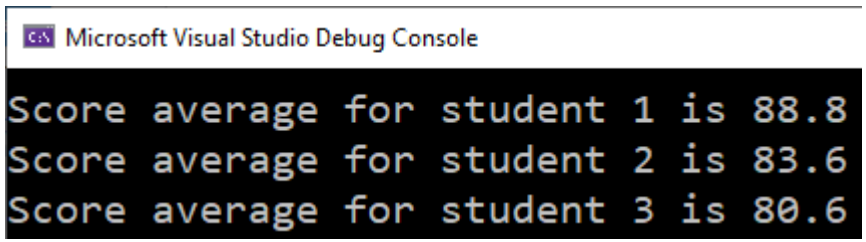
```
// Sum the array elements.
for (int row = 0; row < NUM_STUDENTS; row++)
{
   for (int col = 0; col < NUM_SCORES; col++)
      total += scores[row][col];
}

// Display the sum.
cout << "The total score for all students is " <<
total << endl;
```



Example 2: Summing the columns of the 2D Array

WESTERN SYDNEY UNIVERSITY

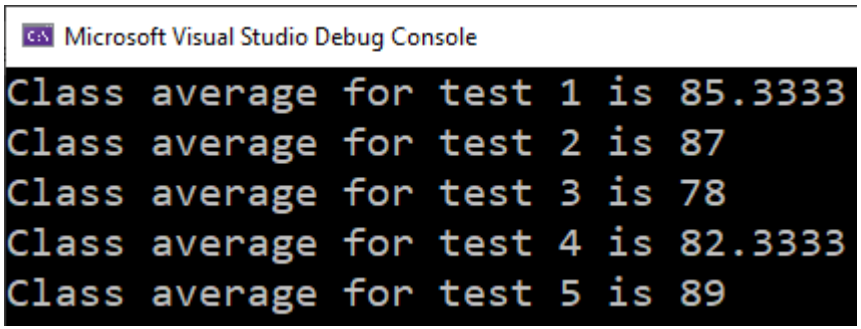
```
// Get each student's average score.
for (int row = 0; row < NUM_STUDENTS; row++)
{
   // Set the accumulator.
   total = 0;
   // Sum a row.
   for (int col = 0; col < NUM_SCORES; col++)
      total += scores[row][col];
   // Get the average
   average = total / NUM_SCORES;
   // Display the average.
   cout << "Score average for student "
        << (row + 1) << " is " << average <<endl;
}
```



Example 3: Summing the rows of the 2D Array

WESTERN SYDNEY UNIVERSITY

```
// Get the class average for each score.
for (int col = 0; col < NUM_SCORES; col++)
{
   // Reset the accumulator.
   total = 0;
   // Sum a column
   for (int row = 0; row < NUM_STUDENTS; row++)
      total += scores[row][col];
   // Get the average
   average = total / NUM_STUDENTS;
   // Display the class average.
   cout << "Class average for test " << (col + 1)
        << " is " << average << endl;
}
```



More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/

WESTERN SYDNEY
UNIVERSITY

2D String Array

ELEC1006: ENGINEERING COMPUTING

2D String Array

WESTERN SYDNEY UNIVERSITY

- Use a two-dimensional array of characters as an array of strings:

```
const int NAMES = 3, SIZE = 10;
char students[NAMES][SIZE] =
 { "Ann", "Bill", "Cindy" };
```

- Each row contains one string

- Can use row subscript to reference the string in a particular row:

```
cout << students[i];
```

Example 1

WESTERN SYDNEY UNIVERSITY

```
1  // This program displays the number of days in each month.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7     const int NUM_MONTHS = 12;  // The number of months
8     const int STRING_SIZE = 10; // Maximum size of each string
9
10    // Array with the names of the months
11    char months[NUM_MONTHS][STRING_SIZE] =
12                  { "January", "February", "March",
13                    "April", "May", "June",
14                    "July", "August", "September",
15                    "October", "November", "December" };
16
17    // Array with the number of days in each month
18    int days[NUM_MONTHS] = {31, 28, 31, 30,
19                            31, 30, 31, 31,
20                            30, 31, 30, 31};
21
22    // Display the months and their numbers of days.
23    for (int count = 0; count < NUM_MONTHS; count++)
24    {
25       cout << months[count] << " has ";
26       cout << days[count] << " days.\n";
27    }
28    return 0;
29 }
```

Example 1 Output

WESTERN SYDNEY UNIVERSITY

Program Output

```
January has 31 days.
February has 28 days.
March has 31 days.
April has 30 days.
May has 31 days.
June has 30 days.
July has 31 days.
August has 31 days.
September has 30 days.
October has 31 days.
November has 30 days.
December has 31 days.
```

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Arrays
https://www.cplusplus.com/doc/tutorial/arrays/

[2] leanrcpp.com: 6.1 – Arrays (Part I)
https://www.learncpp.com/cpp-tutorial/61-arrays-part-i/

[3] learncpp.com: 6.2 – Arrays (Part II)
https://www.learncpp.com/cpp-tutorial/62-arrays-part-ii/

[4] learncpp.com: 6.3 – Arrays and loops
https://www.learncpp.com/cpp-tutorial/63-arrays-and-loops/